

Table of Contents

* Hand Written Calculations

Problem #1 _____ Pgs 2-4

Solutions for a) _____ Pg 3

c) _____ Pg 3

e) _____ Pg 4

{Solution for b), d), f) found in Appendix A}

Problem #2 (Problem statement only) _____ Pg 5

{Solution for problem statement found in Appendix B}

Appendix A _____ Pgs 6-15

Figures _____ Pgs 6-8

Matlab Code _____ Pgs 9-10

Matlab Command Window _____ Pg 11

Simulink Model _____ Pgs 12-15

Appendix B _____ Pgs 16-18

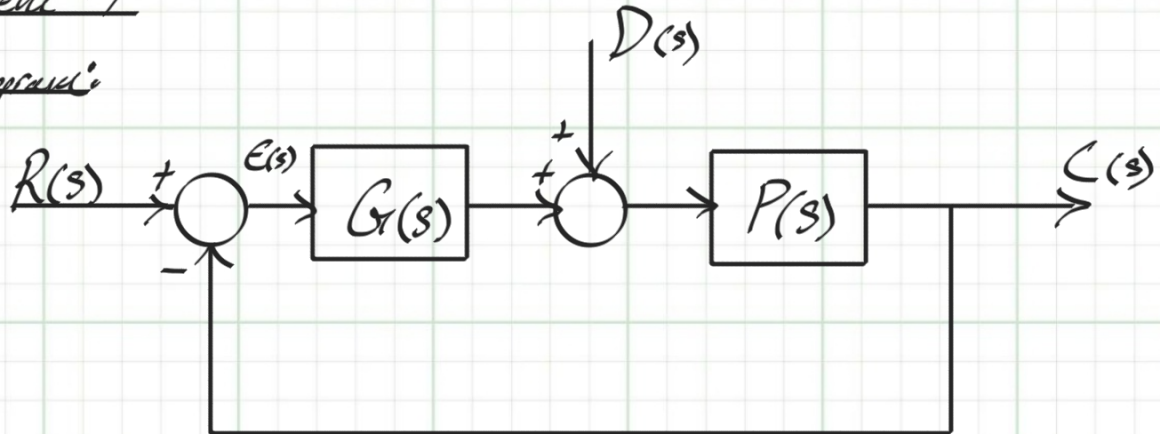
Figures _____ Pg 16

Matlab Code _____ Pg 17

Matlab Command Window _____ Pg 18

Problem #1

Diagram:



Given:

* $G(s) = s$

* $P(s) = \frac{7}{s+2}$

Determine

a) Calculate Steady State error $e_{ss}(t) \Big|_{\substack{t \rightarrow \infty \\ R(s) = \frac{3}{s} \\ D(s) = 0}}$

b) Verify results from a) using simulate

c) Calculate Steady State error $e_{ss}(t) \Big|_{\substack{t \rightarrow \infty \\ R(s) = 0 \\ D(s) = -\frac{1}{s}}}$

d) Verify results from c) using simulate

e) Calculate Steady State error $e_{ss}(t) \Big|_{\substack{t \rightarrow \infty \\ R(s) = \frac{3}{s} \\ D(s) = -\frac{1}{s}}}$

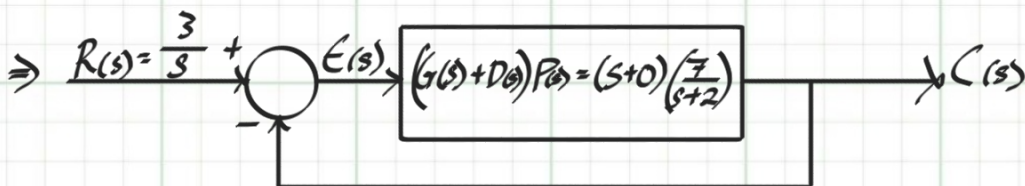
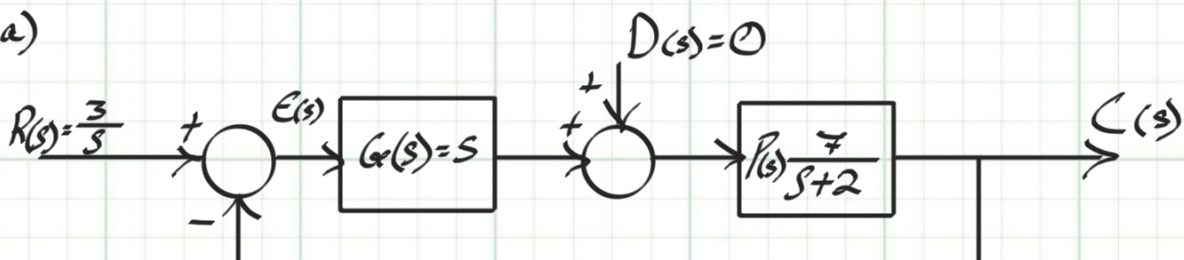
f) Verify results from e) using simulate

Note:

* S.S. error is relative to $R(s)$

Analysis:

a)



$$E(s) = R(s) - C(s) \mid C(s) = \frac{(G(s) + D(s))P(s)}{1 + (G(s) + D(s))P(s)} R(s)$$

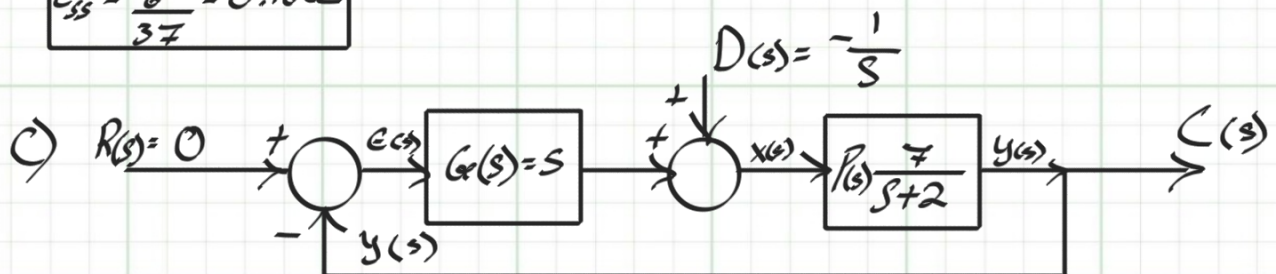
$$\Rightarrow E(s) = \frac{R(s)}{1 + (G(s) + D(s))P(s)} = \frac{\frac{3}{s}}{1 + s \frac{7}{s+2}} = \frac{\frac{3}{s}}{\frac{s+2+3s}{s+2}} = \frac{3(s+2)}{s^2 + 37s}$$

$$E(s) = \frac{3(s+2)}{s(s+37)}$$

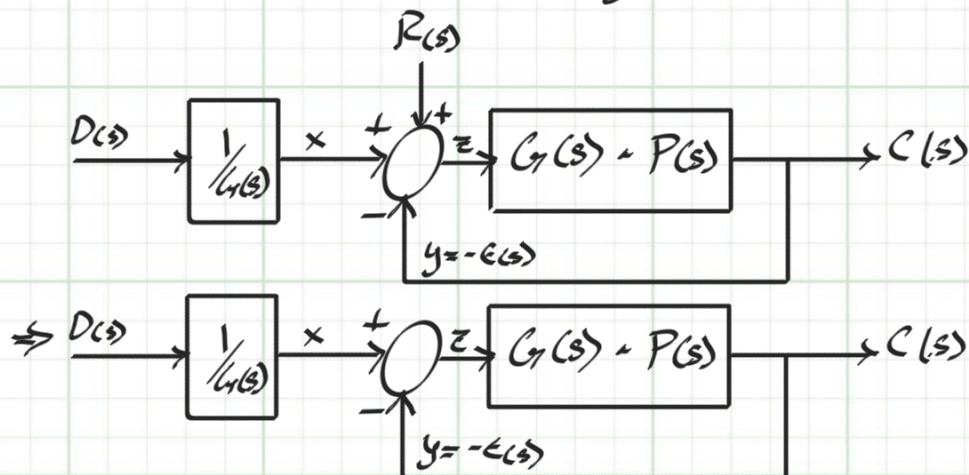
$$e(t) @ (t=\infty) = e_{ss}$$

$$\Rightarrow e_{ss} = \lim_{s \rightarrow 0} s E(s) = \lim_{s \rightarrow 0} s \frac{3(s+2)}{s(s+37)} = \frac{6}{37} = 0.1622$$

$$e_{ss} = \frac{6}{37} = 0.1622$$



$$\therefore E(s) = R(s) - y(s) \therefore E(s) = -y(s) = -C(s)$$



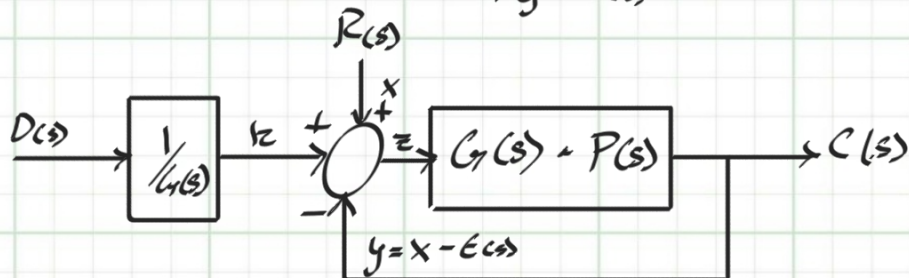
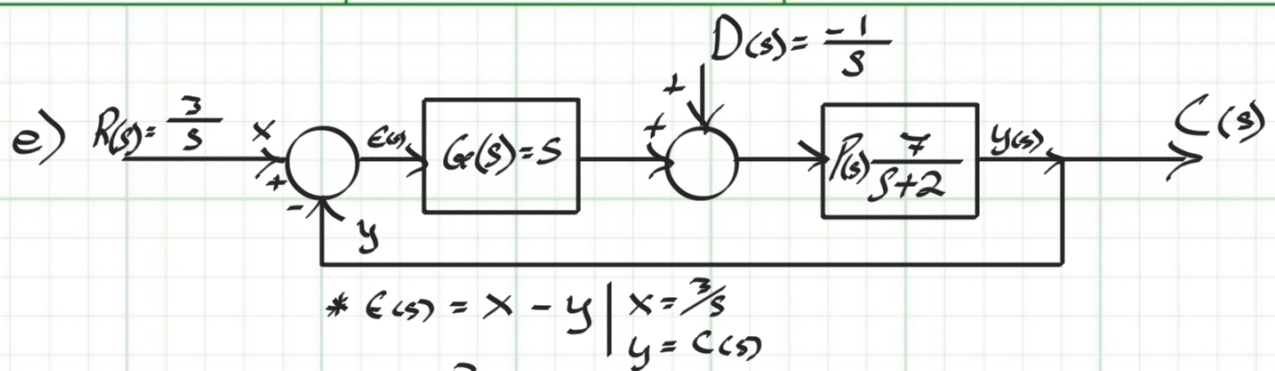
$$\Rightarrow C(s) = \frac{D(s) \cdot G(s)P(s)}{1 + G(s)P(s)} = \frac{D(s)P(s)}{1 + G(s)P(s)} = y = -E(s)$$

$$= \frac{-\frac{1}{s} \left(\frac{7}{s+2} \right)}{1 + \frac{35}{s+2}} \Rightarrow \frac{-7/s}{s+37} = \frac{-7}{s(s+37)}$$

$$E(s) = \frac{7}{s(s+37)}$$

$$e(t) @ (t=\infty) = e_{ss} = \lim_{s \rightarrow 0} s E(s) = \frac{7}{37} = 0.1892$$

$$e_{ss} = \frac{7}{37} = 0.1892$$



$$C(s) = \left(R(s) + \frac{D(s)}{G(s)} \right) \left(\frac{G(s)P(s)}{1 + G(s)P(s)} \right)$$

$$= (R(s)G(s) + D(s)) \frac{P(s)}{1 + G(s)P(s)}$$

$$= \frac{P(s)(R(s)G(s) + D(s))}{1 + G(s)P(s)}$$

$$= \frac{7(1s - 1)}{(s+2)(s)} \times \left(\frac{(s+2)}{s+37} \right) = \frac{98}{s(s+37)} = C(s) = y$$

$$\Rightarrow e(s) = \frac{98}{s(s+37)} - \frac{3}{s} \Rightarrow \frac{98 - 3(s+37)}{(s+37)s} = \frac{-3s - 13}{s(s+37)}$$

$$e(t) @ (t \rightarrow \infty) = e_{ss} = \lim_{s \rightarrow 0} s e(s) = s \left(\frac{-3s - 13}{s(s+37)} \right) = \frac{-13}{37} = -0.3514$$

$$e_{ss} = \frac{-13}{37} = -0.3514$$

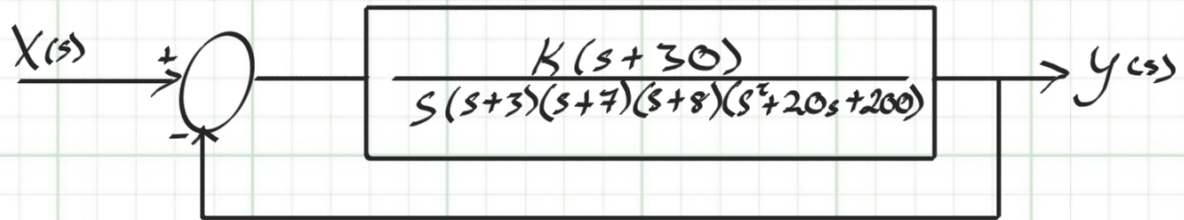
b), d), f) verification results/outputs/simulate models

Shown in Appendix A

(Simulate models for each a), c), & e) validated hand calculated e_{ss} values).

Problem #2

Diagram:



For The following write Program in Matlab

- Display the root locus
- Display close-up using axes limited to $(-2, 2)$ on both real and imaginary axis
- Overlay the 0.707 damping ratio line on the close-up root locus
- On plot interactively select point where the root locus crosses the damping ratio. Give the gain @ this point.
- Generate the step at the gain for the 0.707 damping ratio.

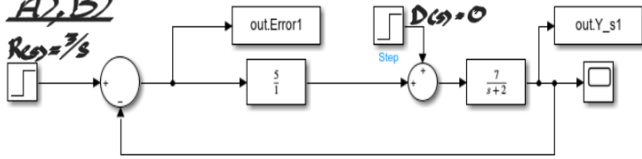
* The following provided in Appendix B

- print out of Matlab Code
- root locus plot
- damping ratio lines
- step response

Appendix A

A), B)

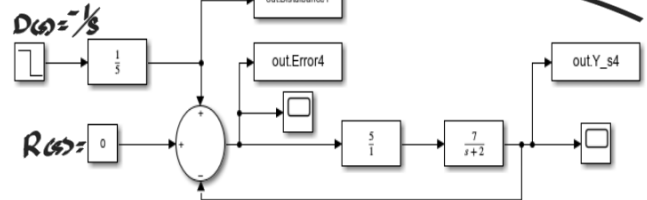
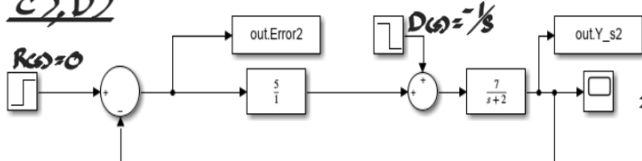
$R(s) = 3/s$



Simple Block diagrams used to show error function is Related to $E(s) = Y(s) - C(s)$ not to location after Summing Junction $R(s) + D(s) - Y(s) = e(s)$

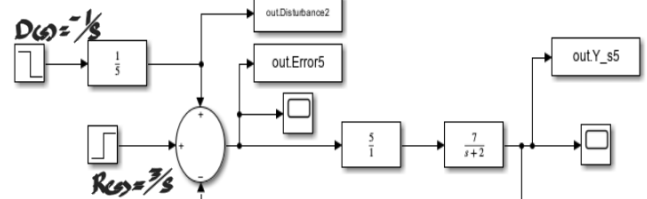
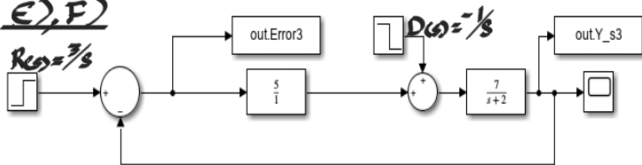
C), D)

$R(s) = 0$

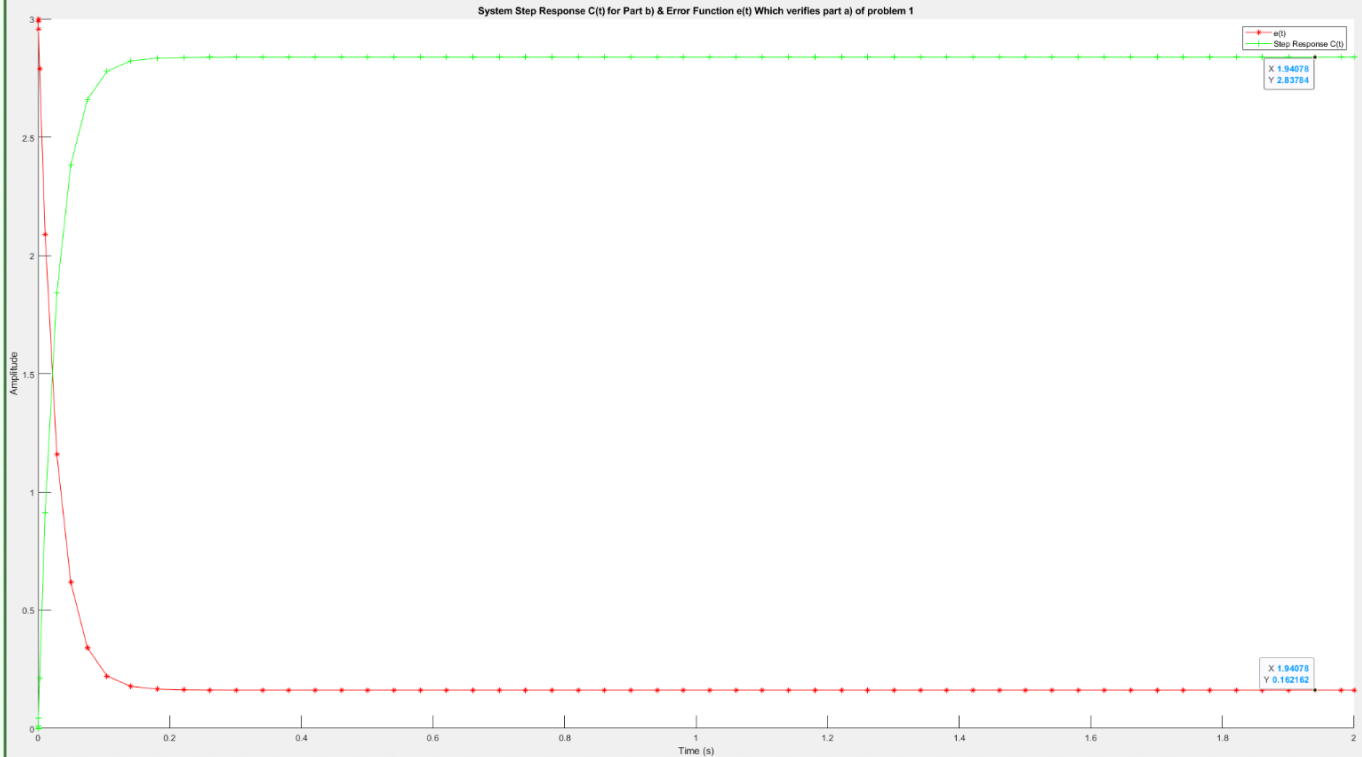


E), F)

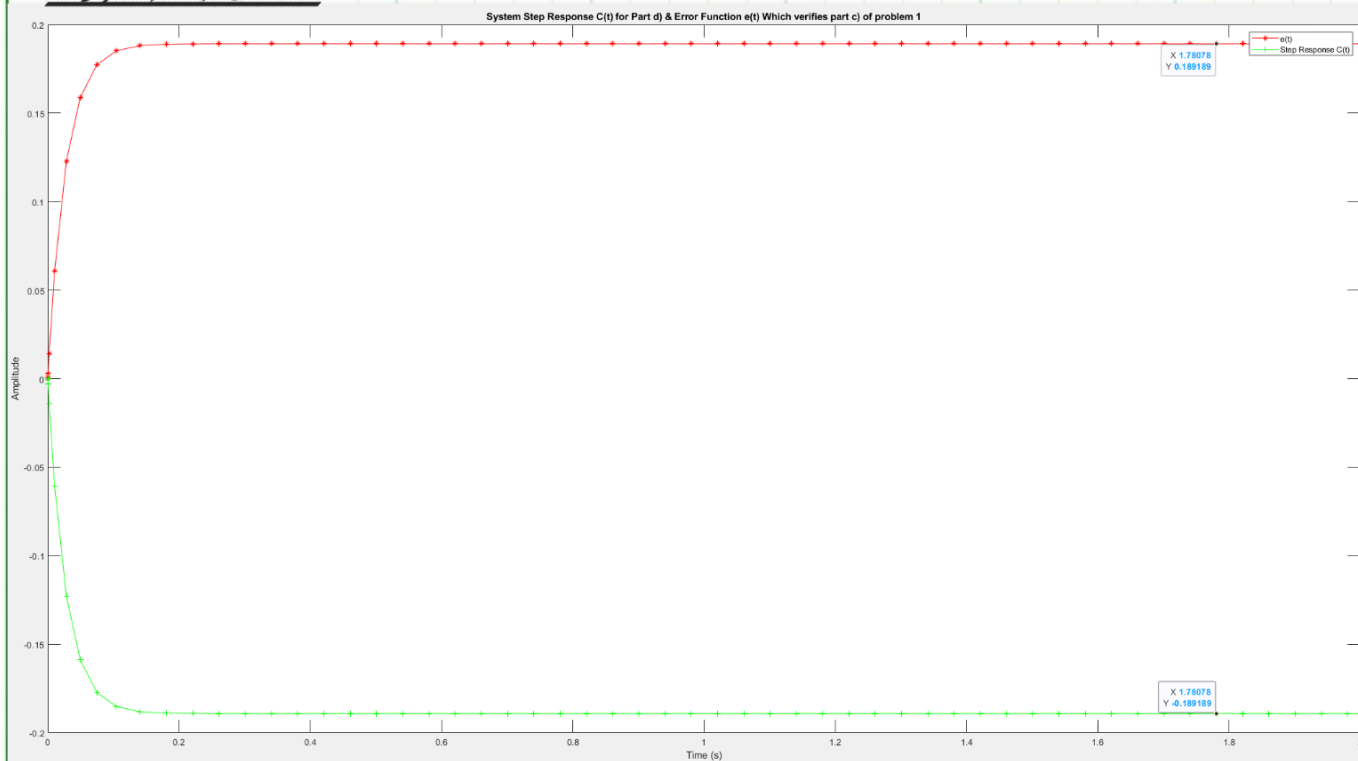
$R(s) = 3/s$



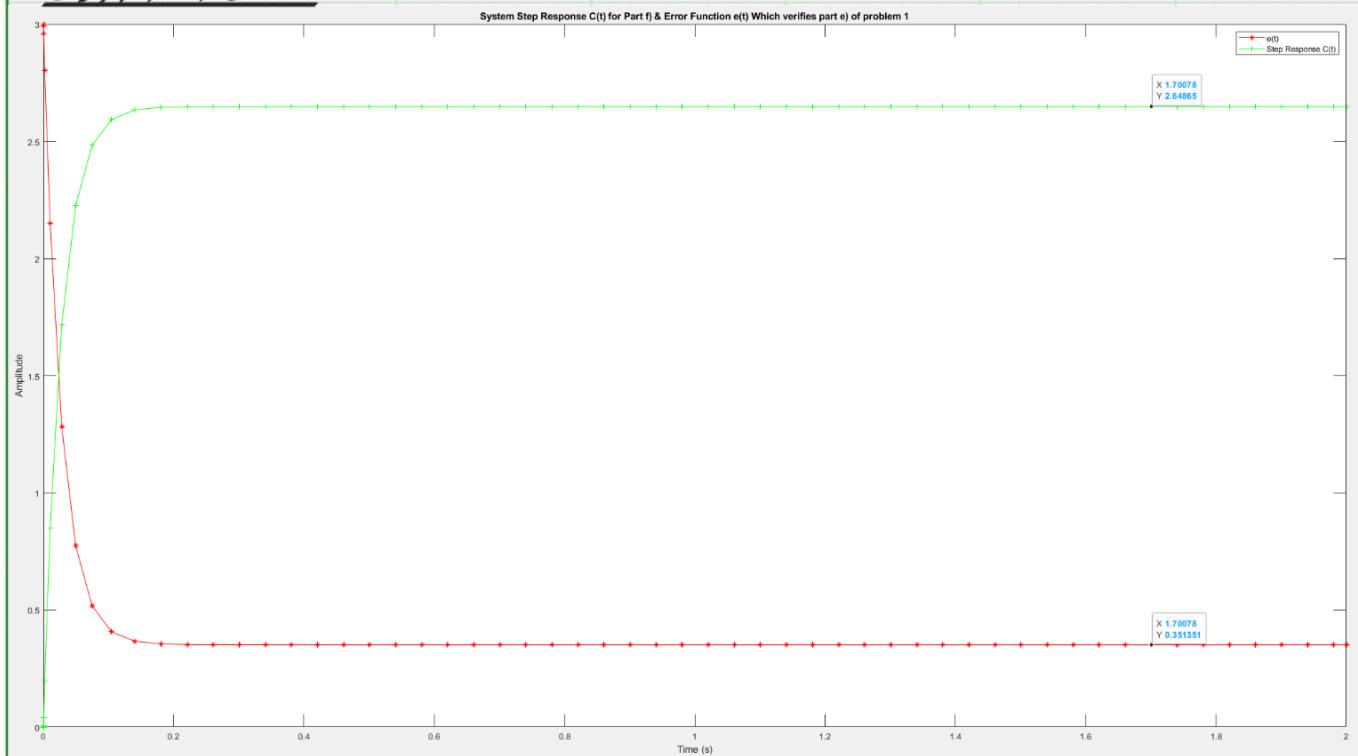
* A), B) Plot



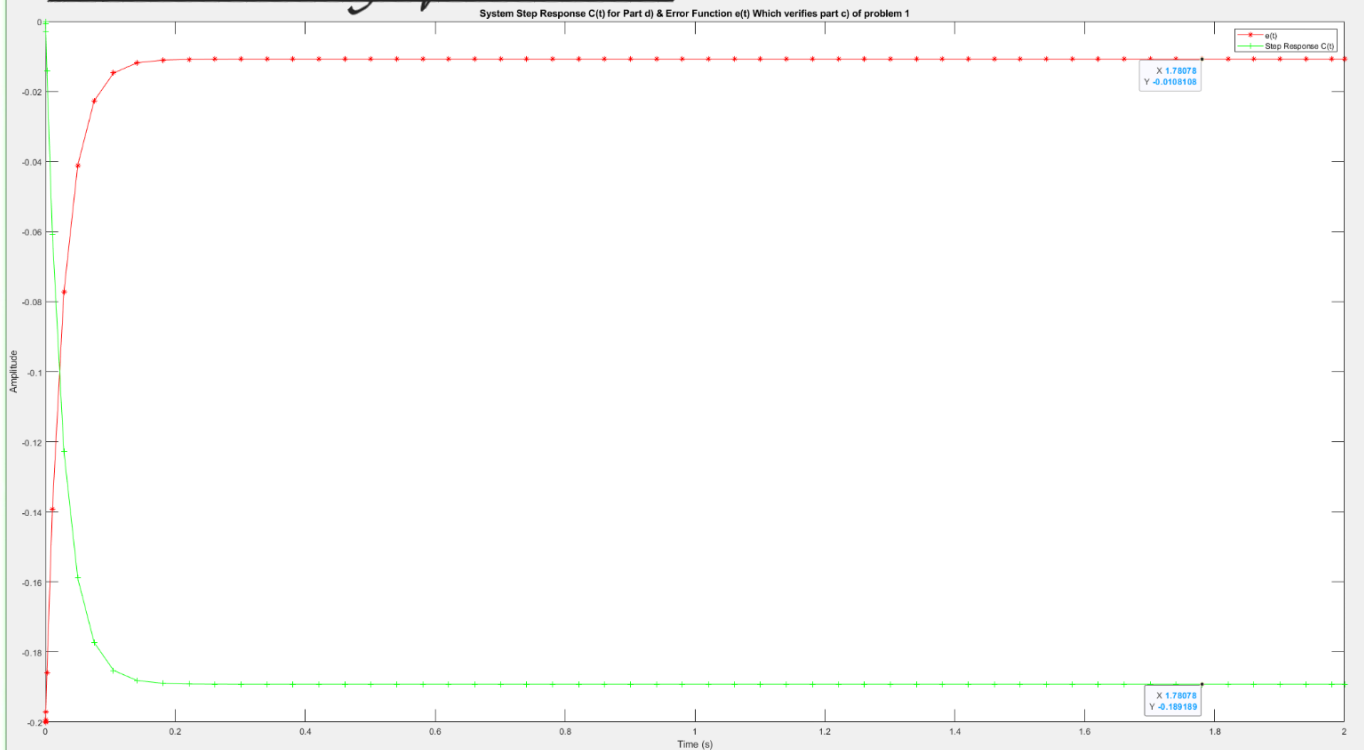
*C),D) Plot



*E),F) Plot



* C), D) Plot Using Simplified Model



- Notice the S.S error shows the error w/ respect to Both R_{ss} + D_{ss}
 this is not the correct error function w/ respect to R_{ss} (shown for completeness and correlation to correct formulation as shown in (* C), D) Plot) figure above.

* E), F) Plot Using Simplified Model



- Notice the S.S error shows the error w/ respect to Both R_{ss} + D_{ss}
 this is not the correct error function w/ respect to R_{ss} (shown for completeness and correlation to correct formulation as shown in (* E), F) Plot) figure above.

```

clc
clear
clf
%{
once block diagrams for a),c),and e) are properly set up in simulinks using
simout blocks at the respective location for error anlysis. Simout block
stores simulink data into the workspace.Save data into accessable file and
then load it back into manipulate.
%}

load("CA1-Pl_Error_out_Data.mat")
Error_Data=[out.Error1.Data,out.Error2.Data,out.Error3.Data,out.Error4.Data,out.Error5.
Data];
Error_time=out.Error1.Time;
Y_s_out=[out.Y_s1.Data,out.Y_s2.Data,out.Y_s3.Data,out.Y_s4.Data,out.Y_s5.Data];
Y_time=out.Y_s1.Time;
Dist_Data=[out.Disturbance1.Data,out.Disturbance2.Data];
%%
%plot the step response data w/ overlay of the error function for anlysis

hold on
problem_Part=["b)" "d)" "f)" "d)" "f)" "a)" "c)" "e)" "c)" "e)"];
for i=1:5
    title_text='System Step Response C(t) for Part '+problem_Part(i)+[' & ' ...
        'Error Function e(t) Which verifies part ']+problem_Part(5+i)+' of problem 1';
    figure(i)
    plot(Error_time, Error_Data(:,i),'r*-',Y_time,Y_s_out(:,i),'g+-')
    title(title_text)
    legend('e(t)','Step Response C(t)')
    xlabel('Time (s)')
    ylabel("Amplitude")
end
%%
% use Error data in comparision of Step Response data from the nonaltered simulink models
to show the Steady
% State error

for i=1:3
    fprintf(['Given Model %s produces a Step Response C(t) with a final value of %f\n' ...
        'Since R(t->inf) reaches a Final Value of %f before input into system\n' ...
        'i.e R(t) @ t=0 (time scale for system)= %f\n' ...
        ' * Implies a Steady State Error e_(ss) = %f w/ respect to R(S) as the input step
function.\n' ...
        ' e_(ss) = R(t = 0) - C(t -> inf) ---> (%f) - (%f) = %f\n\n'] ...
        ,problem_Part(i),Y_s_out(end,i),Error_Data(1,i),Error_Data(1,i),Error_Data(end,i),
Error_Data(1,i), ...
        Y_s_out(end,i),Error_Data(end,i))
end
%%
% use Error data in comparision of Step Response data from the altered/ simplified
simulink models to show the Steady

```

```
% State error in relations to R(s) and only R(s) because simplified models
% alters the linkage were the error is noticably input into the system.
% therefore this is to show that adjusting the models one needs to be aware
% of the data one is directly looking for.
```

```
for i=1:2
    fprintf(['Given Simplified Model %s produces a Step Response C(t) with a ' ...
        'final value of %f and R(t = 0) = %f\n' ...
        'While the Disturbance D(s) into system after simplification produces ' ...
        'a constant value of x(t) = %f into system.\n' ...
        'Implies e_(ss) = R(t = 0) - C(t -> Inf) + x(t) = (%f) - (%f) + (%f) = %f\n\n' ...
        ' * This however is not the same error and is the error with respect ' ...
        'to Both R(s) and D(S) after the summing junction\n' ...
        ' The actual steady state error e_ss w/ respect to R(s) only is ' ...
        'Calculated as follows:\n' ...
        ' e_(ss) = R(t = 0) - C(t -> inf) = (%f) - (%f) = %f\n\n'] ...
        ,problem_Part(1+i),Y_s_out(end,i+3),Error_Data(1,i+1),Dist_Data(1,i),Error_Data(1,
i+1), ...
        Y_s_out(end,i+3),Dist_Data(1,i),Error_Data(end,i+3),Error_Data(1,i+1),Y_s_out(end,
i+3),Error_Data(end,i+1))
end
```


Given Model b) produces a Step Response $C(t)$ with a final value of 2.837838
 Since $R(t \rightarrow \infty)$ reaches a Final Value of 3.000000 before input into system
 i.e $R(t)$ @ $t=0$ (time scale for system) = 3.000000

* Implies a Steady State Error e_{ss} = 0.162162 w/ respect to $R(s)$ as the input step function.

$$e_{ss} = R(t = 0) - C(t \rightarrow \infty) \rightarrow (3.000000) - (2.837838) = 0.162162$$

Given Model d) produces a Step Response $C(t)$ with a final value of -0.189189
 Since $R(t \rightarrow \infty)$ reaches a Final Value of 0.000000 before input into system
 i.e $R(t)$ @ $t=0$ (time scale for system) = 0.000000

* Implies a Steady State Error e_{ss} = 0.189189 w/ respect to $R(s)$ as the input step function.

$$e_{ss} = R(t = 0) - C(t \rightarrow \infty) \rightarrow (0.000000) - (-0.189189) = 0.189189$$

Given Model f) produces a Step Response $C(t)$ with a final value of 2.648649
 Since $R(t \rightarrow \infty)$ reaches a Final Value of 3.000000 before input into system
 i.e $R(t)$ @ $t=0$ (time scale for system) = 3.000000

* Implies a Steady State Error e_{ss} = 0.351351 w/ respect to $R(s)$ as the input step function.

$$e_{ss} = R(t = 0) - C(t \rightarrow \infty) \rightarrow (3.000000) - (2.648649) = 0.351351$$

Given Simplified Model d) produces a Step Response $C(t)$ with a final value of -0.189189 and $R(t = 0) = 0.000000$

While the Disturbance $D(s)$ into system after simplification produces a constant value of $x(t) = -0.200000$ into system.

Implies $e_{ss} = R(t = 0) - C(t \rightarrow \infty) + x(t) = (0.000000) - (-0.189189) + (-0.200000) = -0.010811$

* This however is not the same error and is the error with respect to Both $R(s)$ and $D(s)$ after the summing junction

The actual steady state error e_{ss} w/ respect to $R(s)$ only is Calculated as follows:

$$e_{ss} = R(t = 0) - C(t \rightarrow \infty) = (0.000000) - (-0.189189) = 0.189189$$

Given Simplified Model f) produces a Step Response $C(t)$ with a final value of 2.648649 and $R(t = 0) = 3.000000$

While the Disturbance $D(s)$ into system after simplification produces a constant value of $x(t) = -0.200000$ into system.

Implies $e_{ss} = R(t = 0) - C(t \rightarrow \infty) + x(t) = (3.000000) - (2.648649) + (-0.200000) = 0.151351$

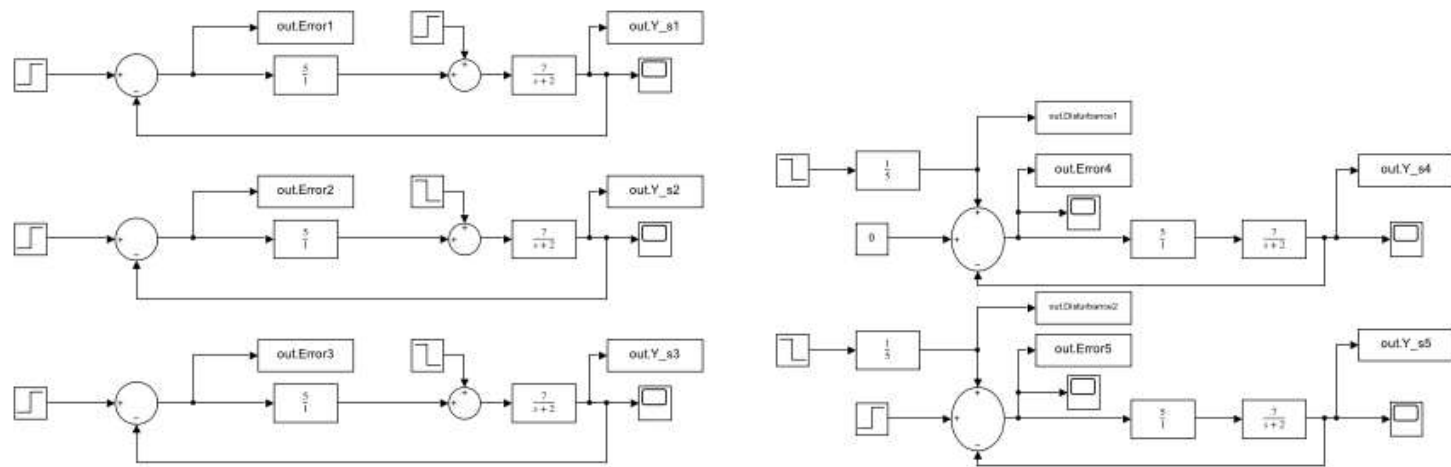
* This however is not the same error and is the error with respect to Both $R(s)$ and $D(s)$ after the summing junction

The actual steady state error e_{ss} w/ respect to $R(s)$ only is Calculated as follows:

$$e_{ss} = R(t = 0) - C(t \rightarrow \infty) = (3.000000) - (2.648649) = 0.351351$$

>>

Problem1_CA2



Jarel Shahim

15-Nov-2023 07:46:56

Table of Contents

[Model - Problem1_CA2](#)

[Machine - Problem1_CA2](#)

[System - Problem1_CA2](#)

[Appendix](#)

List of Tables

- [1. Constant Block Properties](#)
- [2. Step Block Properties](#)
- [3. Sum Block Properties](#)
- [4. ToWorkspace Block Properties](#)
- [5. TransferFcn Block Properties](#)
- [6. Block Type Count](#)

Model - Problem1_CA2

Table of Contents

[Machine - Problem1_CA2](#)

Full Model Hierarchy

- [1. Problem1_CA2](#)

Simulation Parameter	Value
Solver	VariableStepAuto
RelTol	1e-3
Refine	1
MaxOrder	5
ZeroCross	on

[\[more info\]](#)

Machine - Problem1_CA2

[\[more info\]](#)

System - Problem1_CA2

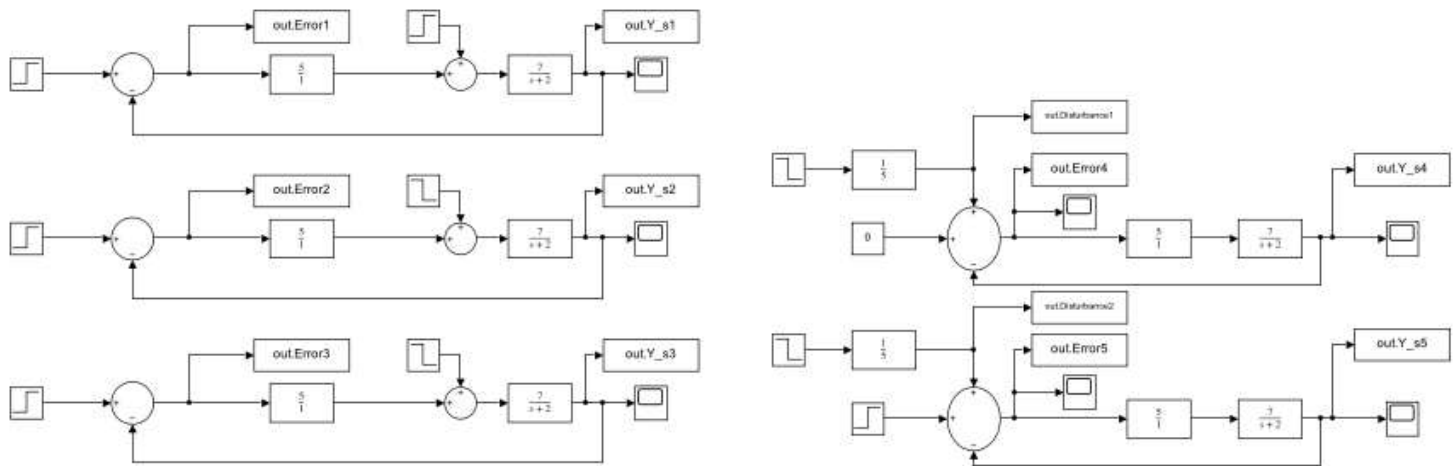


Table 1. Constant Block Properties

Name	Value	Out Data Type Str	Lock Scale	Sample Time	Frame Period
Constant1	0	Inherit: Inherit from 'Constant value'	off	inf	inf

Table 2. Step Block Properties

Name	Time	Before	After	Out Data Type Str	Sample Time	Zero Cross
Step	0	0	0	double	0	on
Step1	0	0	-1	double	0	on
Step2	0	0	3	double	0	on
Step3	0	0	-1	double	0	on
Step4	0	0	0	double	0	on
Step5	0	0	-1	double	0	on
Step6	0	0	3	double	0	on
Step7	0	0	-1	double	0	on
Step8	0	0	3	double	0	on

Table 3. Sum Block Properties

Name	Icon Shape	Inputs	Collapse Mode	Collapse Dim	Out Data Type Str	Accum Data Type Str	Input Same DT	Lock Scale	Rnd Meth	Saturate On Integer Overflow
Sum	round	++	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum1	round	+-	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off

Name	Icon Shape	Inputs	Collapse Mode	Collapse Dim	Out Data Type Str	Accum Data Type Str	Input Same DT	Lock Scale	Rnd Meth	Saturate On Integer Overflow
Sum2	round	++	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum3	round	+ + -	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum4	round	+-	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum5	round	++	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum6	round	+-	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off
Sum7	round	+ + -	All dimensions	1	Inherit: Inherit via internal rule	Inherit: Inherit via internal rule	off	off	Floor	off

Table 4. ToWorkspace Block Properties

Name	Variable Name	Max Data Points	Decimation	Save Format	Save 2DSignal	Fixpt As Fi
To Workspace	Error1	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace1	Y_s1	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace10	Disturbance1	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace11	Disturbance2	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace2	Error4	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace3	Y_s4	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace4	Error2	inf	1	Timeseries	3-D array (concatenate along third dimension)	on

Name	Variable Name	Max Data Points	Decimation	Save Format	Save 2DSignal	Fixpt As Fi
To Workspace5	Y_s2	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace6	Error3	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace7	Y_s3	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace8	Error5	inf	1	Timeseries	3-D array (concatenate along third dimension)	on
To Workspace9	Y_s5	inf	1	Timeseries	3-D array (concatenate along third dimension)	on

Table 5. TransferFcn Block Properties

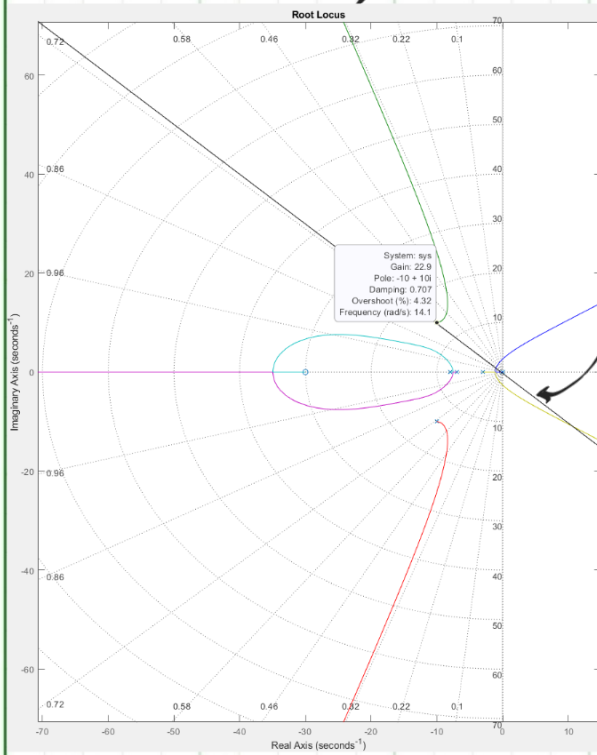
Name	Numerator	Denominator	Parameter Tunability	Absolute Tolerance	Continuous State Attributes
Transfer Fcn	5	1	Auto	auto	"
Transfer Fcn1	7	[1 2]	Auto	auto	"
Transfer Fcn10	7	[1 2]	Auto	auto	"
Transfer Fcn11	1	5	Auto	auto	"
Transfer Fcn2	7	[1 2]	Auto	auto	"
Transfer Fcn3	1	5	Auto	auto	"
Transfer Fcn4	5	1	Auto	auto	"
Transfer Fcn5	5	1	Auto	auto	"
Transfer Fcn6	7	[1 2]	Auto	auto	"
Transfer Fcn7	5	1	Auto	auto	"
Transfer Fcn8	7	[1 2]	Auto	auto	"
Transfer Fcn9	5	1	Auto	auto	"

Appendix

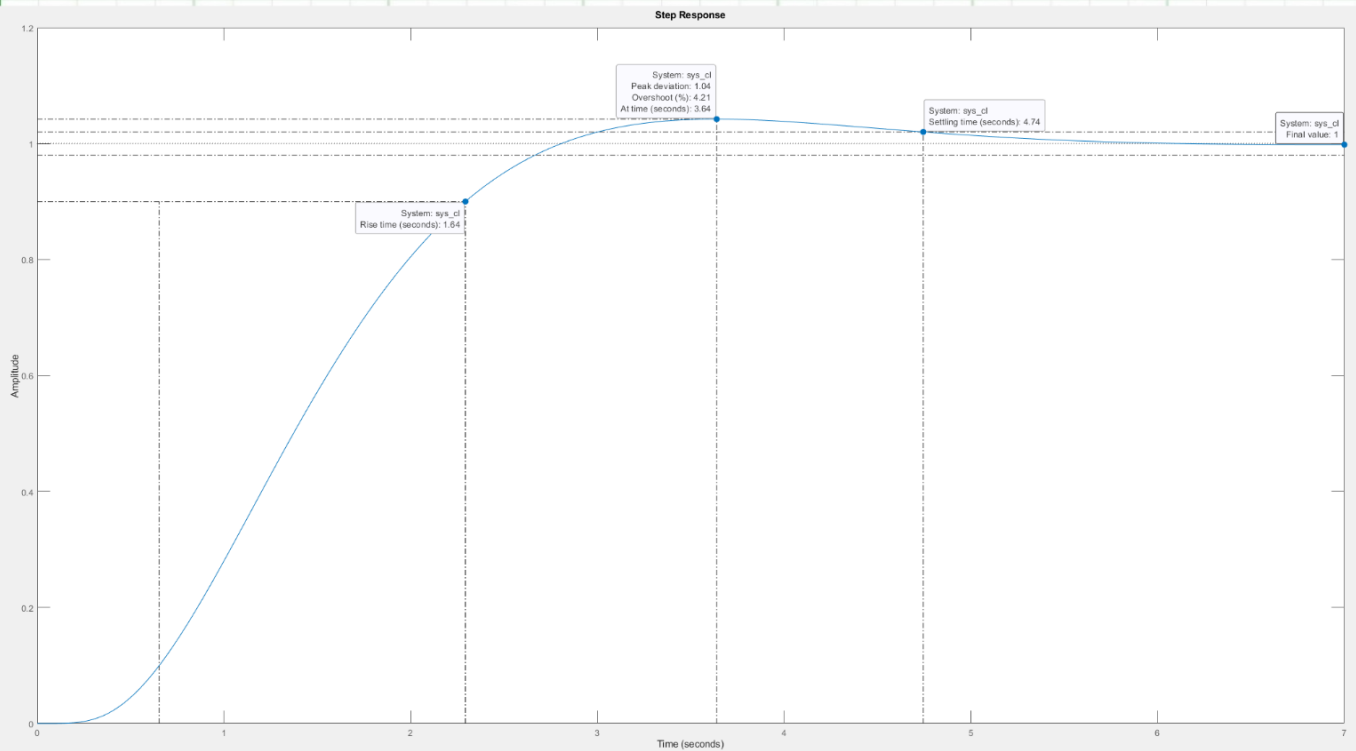
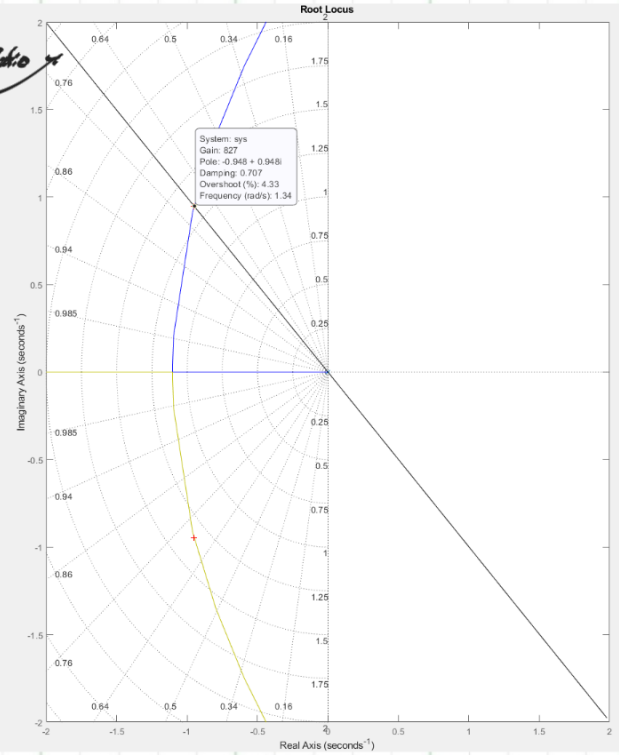
Table 6. Block Type Count

BlockType	Count	Block Names
TransferFcn	12	Transfer Fcn , Transfer Fcn1 , Transfer Fcn10 , Transfer Fcn11 , Transfer Fcn2 , Transfer Fcn3 , Transfer Fcn4 , Transfer Fcn5 , Transfer Fcn6 , Transfer Fcn7 , Transfer Fcn8 , Transfer Fcn9
ToWorkspace	12	To Workspace , To Workspace1 , To Workspace10 , To Workspace11 , To Workspace2 , To Workspace3 , To Workspace4 , To Workspace5 , To Workspace6 , To Workspace7 , To Workspace8 , To Workspace9
Step	9	Step , Step1 , Step2 , Step3 , Step4 , Step5 , Step6 , Step7 , Step8
Sum	8	Sum , Sum1 , Sum2 , Sum3 , Sum4 , Sum5 , Sum6 , Sum7
Scope	7	Scope, Scope1, Scope2, Scope3, Scope4, Scope5, Scope6
Constant	1	Constant1

Appendix B



Damping Ratio Line



```
clear
clc
clf
%%---Run One section at a time---%%

%set up transfer function
s=tf('s');
sys=(s+30)/(s*(s+3)*(s+7)*(s+8)*(s^2+20*s+200));

%set up axis system limits with the first indexing set so it makes
%overlaying the damping ratio line easier. set the second index to the
%zoomed in parameters for the root locus

axis_lim=[[-70.7,15,-70.7,70.7];[-2,2,-2,2]];

% set the perscribed damping ratio

Damp_ratio=0.707;

%Plot the root locus in subplots using rlocus() with an sgrid() overlay w/
%respect to the dampio ratio. this grid w/ an overlayed(manually) damping
%ratio will help in determining real values of gain that would stablize the
%sytem into an underdamped transform.

hold on
figure(1)
for i=1:2
    subplot(1,2,i)
    sgrid(Damp_ratio)
    rlocus(sys)
    axis(axis_lim(i,:))
j=i;
end
%%
%%--- Run section---%%

%Using Rlocfind lets you manually anaylize point locations on the root
%locus, pick location were the damping ratio overlayed line crosses with
%the root locus on the zoomed plot. k= gain
[k ,poles]=rlocfind(sys)
%%
%%---Run Section---%%

%use feedback(T(s),H(s)) function to compile the (sys*k)=T(s) with H(s)=1
% into the desired closed loop. use step()function to produce the stable underdamped
% step response.
sys_cl=feedback(k*sys,1)
figure(2)
step(sys_cl)
```

Select a point in the graphics window

selected_point =

-0.9467 + 0.9515i

k =

828.8086

poles =

-9.9500 +10.0085i

-9.9500 -10.0085i

-8.1011 + 1.8585i

-8.1011 - 1.8585i

-0.9489 + 0.9522i

-0.9489 - 0.9522i

sys_cl =

828.8 s + 2.486e04

s^6 + 38 s^5 + 661 s^4 + 5788 s^3 + 23560 s^2 + 3.443e04 s + 2.486e04

Continuous-time transfer function.

Model Properties

>>